



Bangladesh University of Engineering and Technology

Course Name: EEE 212

Course Title : Numerical Technique Laboratory.

Project Title: Understanding the Butterfly Effect with Double Pendulum.

Submitted By:

Md. Maksudur Rahman Turzo - 2006053

Joyonta Kartik Debnath - 2006054

Group: 10

Section: A2

L2-T1

Submitted To:

Bejoy Sikdar

Lecturer, Dept. of EEE, BUET

Problem statement: Understanding the Butterfly effect and chaos theory through the double pendulum simulation.

Introduction: Butterfly effect is the idea that a small amount of initial change can cause a large deviation or consequence later on. This idea can be properly simulated or proved through a double pendulum experiment. In this project we tried to simulate the motion of a double pendulum as well as change the initial conditions slightly and check if the theory holds. In order to do this, the lagrangian form of motion of a double pendulum was used. This second order simultaneous ODE was solved numerically and with built-in function of Matlab. This gave us the simulation of a double pendulum. Also, we used euler and improved euler method to compare between the graphs. From the simulation it was clear that there was error in the euler and improved euler method.

Objective:

The main objective of this Project is about Visual Understanding of the Butterfly Effect. For the project we have used the system of a double pendulum. In detail the objectives of this project is --

- Solving Lagrangian by in built matlab function and Iterative methods.
- Learning about motion of a Double Pendulum.
- Visual understanding of the Chaos theory.
- Chaos caused by the motion of Double Pendulum.
- Understanding The Butterfly Effect caused by different initial conditions of a Chaotic motion.
- Difference of different iterative methods of solving ODE.

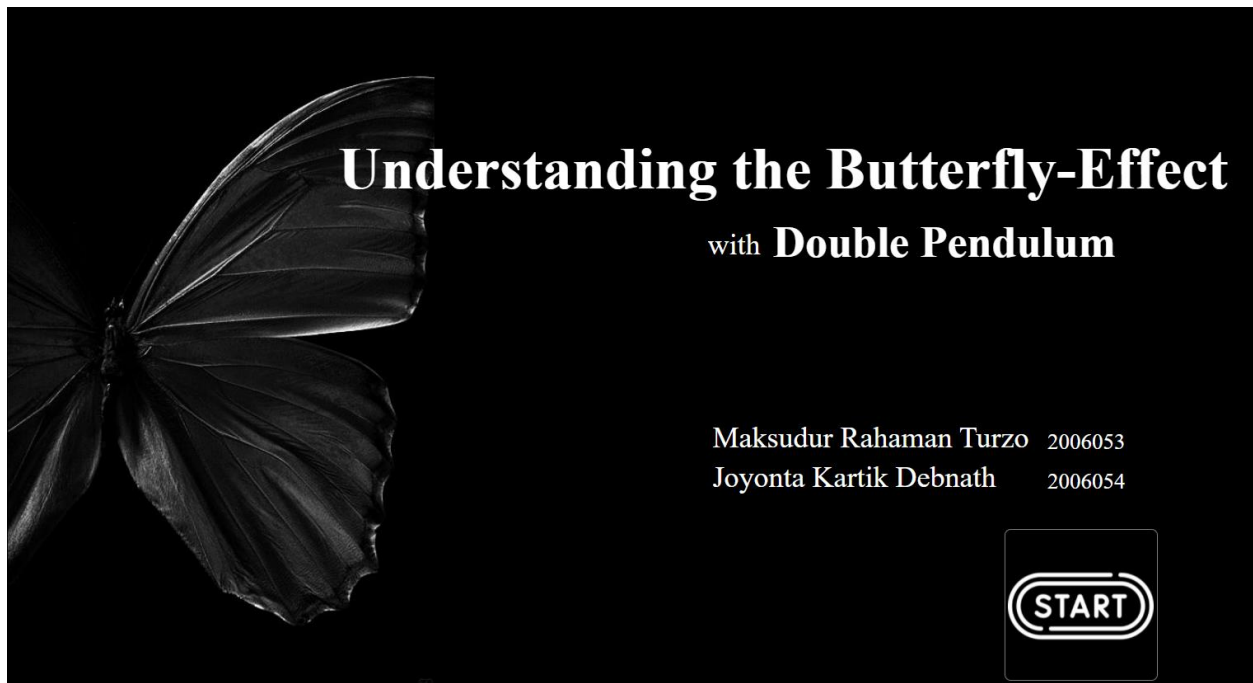
- Understanding The Butterfly Effect caused by errors of different type of Iterative methods.

Achieved Outcome:

The app contains multiple tabs. each tabs are described below.

Tab 1: Introduction page --

This is just a cover page of the app. This contains the project name and the developers name.



Tab 2: Theoretical Discussion --

In this page a brief theory about the project was added. So that if someone wants to run the program and don't know about the theories he can get a short idea about it. In this part short discussions about double pendulum, chaos theory and Butterfly effect were added for future users.

Theory:

The Butterfly Effect:

The butterfly effect is a concept in chaos theory that refers to the idea that small changes in initial conditions can lead to vastly different outcomes in a system over time. The term was coined by Edward Lorenz, a mathematician who discovered the effect while studying weather patterns.

The butterfly effect suggests that a small change in one part of a system can have a large effect on another part of the system, even if the connection between the two is not immediately obvious. This is because complex systems, such as weather patterns or ecosystems, are often nonlinear and highly sensitive to initial conditions.

Examples:

Weather Patterns:

The butterfly effect was discovered by Edward Lorenz while studying weather patterns. He found that small changes in the initial conditions of a weather model, such as rounding a value to a different number of decimal places, could result in significantly different weather patterns over time. For example, a butterfly flapping its wings in Brazil could set off a chain of events that leads to a tornado in Texas.

Traffic Jams:

Traffic jams are another example of the butterfly effect. A small delay, such as a car stopping suddenly or a traffic light changing too quickly, can lead to a chain reaction that causes a traffic jam. This is because cars on the road are constantly adjusting their speed and distance to other cars, and a small change in one car's behavior can have a ripple effect throughout the entire system.

Ecology:

The butterfly effect is also evident in ecological systems. A small change in one part of an ecosystem, such as the introduction of a new species or the removal of a predator, can have far-reaching effects on other species and the overall balance of the ecosystem.

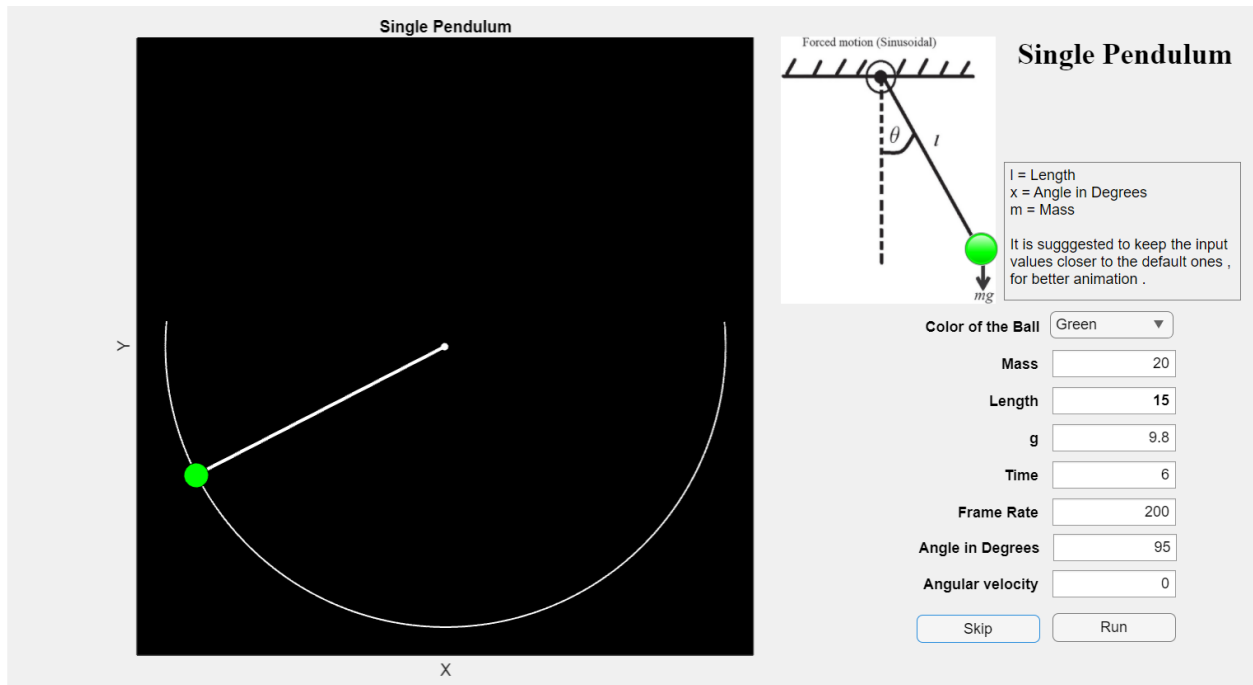
Stock Market:

The stock market is another example of the butterfly effect in action. A small change in the value of one stock can have a ripple effect throughout the market, causing prices to rise or fall and affecting the overall economy.

The butterfly effect is a fascinating concept that demonstrates the interconnectedness and complexity of the world around us. Its

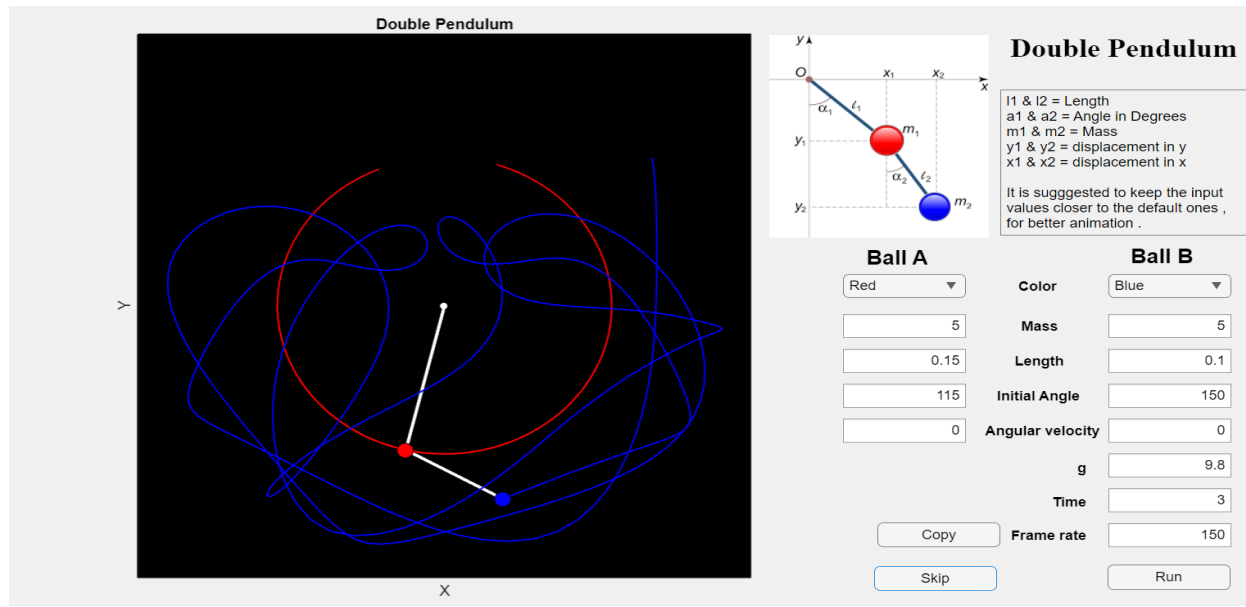
Tab 3: Single pendulum --

In this page a simple pendulum simulation is added. In the right side the users can give input about the initial conditions of the simple pendulum. And see the simulation of the motion of the simple pendulum according to time. The users can also select the color of the pendulum and also a skip button was added if the user wishes to skip to the last part of the motion and wants to see the full trajectory.



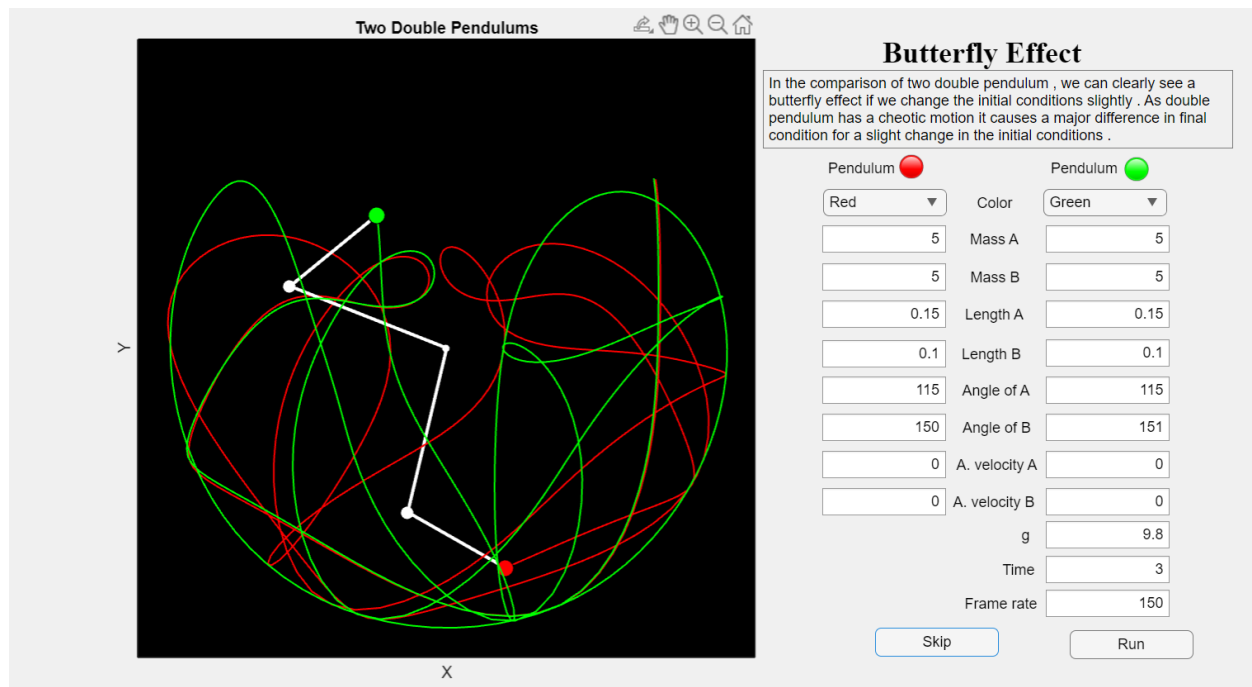
Tab 4: Double pendulum --

This is where things get interesting. In this tab users can give the initial conditions of a double pendulum and see trajectory for both of the balls at the same time and also in this tab too we can select the color of the balls and the color of the trajectory. and also skip to the end if we want to skip the animation and just see the whole trajectory of the pendulums.



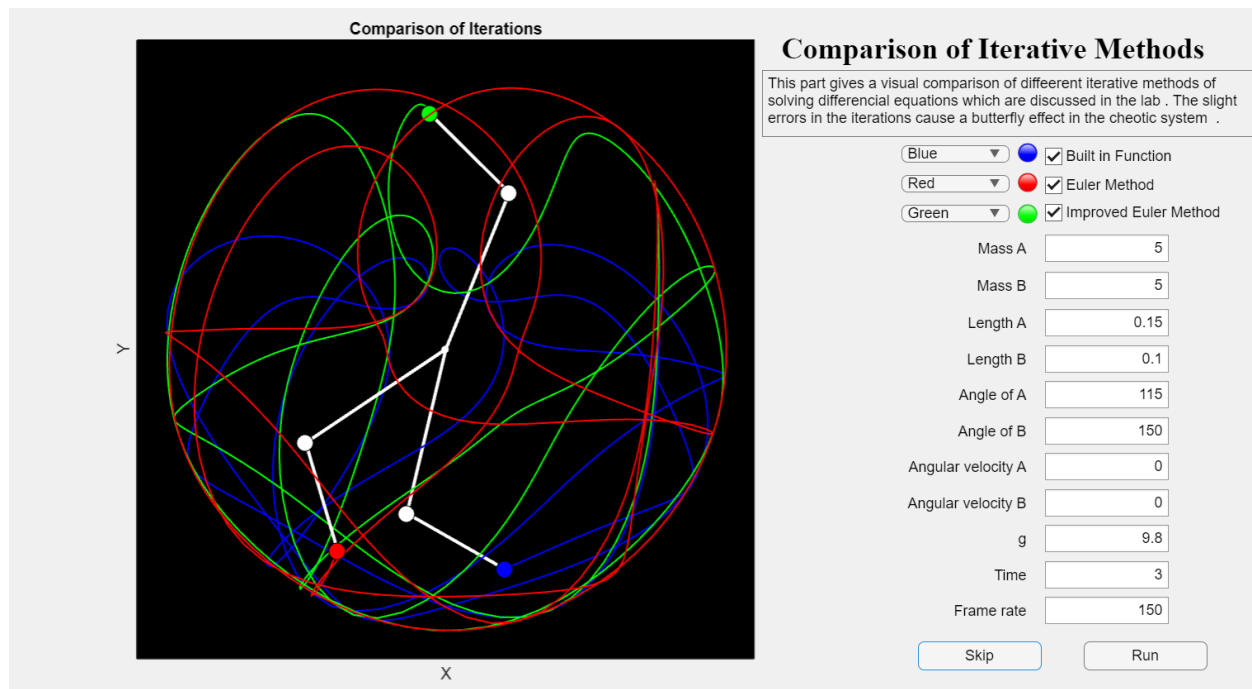
Tab 5: Two double pendulums & butterfly effect --

In this tab we can compare between two double pendulums by changing their initial conditions and see the butterfly effect caused by them. Though we can change as much as we want but it is suggested to change the initial conditions only slightly to observe the butterfly effect. In this tab too we can select the color of the balls and the color of the trajectory and also skip to the end if we want to skip the animation and just see the whole trajectory of the pendulums.



Tab 6: Comparison of iterative methods --

In the lab we learnt 2 iterative methods to solve ODE . Both of them had slight errors. In this page we can visualize that how these slight errors can cause butterfly effect and cause a huge change in the final conditions of a chaotic motion. 3 methods of analysis were added, one is the inbuilt function , eulers method and improved eulers method.



Limitations:

- The first limitation of the project is the execution time. Though the execution time may vary from machine to machine, but in the developers computer the program takes some seconds to execute.
- Another limitation is the skip button. In the program there are pendulum motion animated and plotted in different places. However if one wants to skip the animation and wants to see the last condition it is possible. A skip button was added in every plot. But the skip button works very slow. In the single pendulum the skip button works almost in no time. But in the other plots such as for the 2 double pendulum and the 1 double pendulum it takes almost 10 seconds to execute skip callback.
- The project gives the best plot for values around the default initials. such as in the default initials the length of the rod is in one tenth of a meter range. But if one wants to give the range of the rod in meter range the system will work but the system will be slower. Because in reality the bigger rod will take a longer time to rotate. Though both plots are accurate but the later one may seem to look less interesting.

- In the Lagrangian of the pendulums the damping of the system was ignored. Because the damping constant made the equation a lot more difficult. So, because of damping no matter what the final condition of the system, after a long time would be same, that is zero displacement. So by considering damping the Double pendulum model can not be considered a butterfly effect in a longer time. So damping was ignored.

Matlab Code:

Code for the inbuilt function to solve ODE:

```

methods (Access = private)
    function dx = double_pendulum_model(app,t,y,parameters)

    mA = parameters(1);           % Mass A           [kg]
    mB = parameters(2);           % Mass B           [kg]
    lA = parameters(3);           % Length rod A      [m]
    lB = parameters(4);           % Length rod B      [m]
    g  = parameters(5);           % Gravity           [m/s2]

    y1 = y(1);
    y2 = y(2);
    y3 = y(3);
    y4 = y(4);

    out(1) = y2;
    out(2) = ( (mA+mB)*lA - mB*lA*(cos(y1-y3))^2 )...
        ^(-1) * ( -mB*lA*(y2^2)*sin(y1-y3)*cos(y1-y3) ...
        + mB*g*sin(y3)*cos(y1-y3) - mB*lB*(y4^2)*sin(y1-y3)...
        - (mA+mB)*g*sin(y1) );
    out(3) = y4;
    out(4) = ( lB - (mB*lB/(mA+mB))*(cos(y1-y3))^2 )...
        ^(-1) * ( lA*(y2)^2*sin(y1-y3) - g*sin(y3) + ...
        (mB*lB/(mA+mB))*(y4^2)* sin(y1-y3)*cos(y1-y3) ...
        + g*sin(y1)*cos(y1-y3) );

    dx = [out(1) ; out(2) ; out(3) ; out(4)];
end

function [t , results] = solution_to_double_pendulam...
    (app,mA , mB , lA , lB , g , theta_A0 , theta_B0 ,...
    velocity_A0 , velocity_B0 , tf , fR )

```

```

parameters = [mA mB lA lB g];
theta_0 = [theta_A0 velocity_A0 theta_B0 velocity_B0];

dt      = 1/fR;
time_span = 0 : dt : tf ;

[ t , results ] = ode45(@(time,theta) double_pendulum_model...
    (app,time,theta,parameters),time_span,theta_0);

end
end

```

Code for the single pendulum:

```

g = app.gEditField.Value;
l = app.LengthEditField.Value;
m = app.MassEditField.Value ;
t = app.TimeEditField.Value;
frame = app.FrameRateEditField.Value;
theta = app.AngleinDegreesEditField.Value*pi/180;
omega = app.AngularvelocityEditField.Value;
theta_0 = [theta ; omega];
c = app.ColoroftheBallDropDown.Value;
app.Lamp.Color = c ;

t_span = linspace(0 , t , frame*t);
[t,results] = ode45(@(t,theta)ode_func(t,theta,g,l,m)...
    ,t_span,theta_0);

x = l*sin(results(:,1));
y = -l*cos(results(:,1));

function [dtheta_dt] = ode_func(t,theta,g,l,m)
    theta1 = theta(1);
    theta2 = theta(2);
    dtheta1_dt = theta2;
    dtheta2_dt = -(g/l)*sin(theta1);
    dtheta_dt = [dtheta1_dt;dtheta2_dt];
end

```

Code for the motion of the double pendulum:

- By inbuilt function:

```

mA = app.MassEditField_2.Value;
mB = app.MassEditField_3.Value;

```

```

rA = app.LengthAEditField.Value;
rB = app.LengthEditField_2.Value;
g = app.gEditField_2.Value;
theta_A0 = app.AngleofAEditField.Value*pi/180;
theta_B0 = app.InitialAngleEditField.Value*pi/180;
velocity_A0 = app.AngularvelocityAEditField.Value;
velocity_B0 = app.AngularvelocityEditField_2.Value;
time = app.TimeEditField_2.Value;
framerate = app.FramerateEditField.Value;
%% process part
[t,results] = solution_to_double_pendulam(app,mA,mB,rA,rB,g,theta_A0,...
    theta_B0,velocity_A0,velocity_B0,time,framerate);

theta_A = results(:,1);
velocity_A = results(:,2);
theta_B = results(:,3);
velocity_B = results(:,4);
xA = rA*sin(theta_A);
yA = -rA*cos(theta_A);
xB = rA*sin(theta_A) + rB*sin(theta_B);
yB = -rA*cos(theta_A) - rB*cos(theta_B);

```

- By eulers iterative method:

```

mA = app.MassAEditField_6.Value;
mB = app.MassBEditField_6.Value;
rA = app.LengthAEditField_6.Value;
rB = app.LengthBEditField_6.Value;
g = app.gEditField_4.Value;

theta_A0 = app.AngleofAEditField_6.Value*pi/180;
theta_B0 = app.AngleofBEditField_6.Value*pi/180;
velocity_A0 = app.AngularvelocityAEditField_2.Value;
velocity_B0 = app.AngularvelocityBEditField_2.Value;

time = app.TimeEditField_4.Value;
framerate = app.FramerateEditField_3.Value;
h = 1/framerate;

y1(1) = theta_A0;
y2(1) = velocity_A0;
y3(1) = theta_B0;
y4(1) = velocity_B0;

t = 0 : h : time ;

```

```

f2 = @(mA,mB,rA,rB,g,y1,y2,y3,y4) ( (mA+mB)*rA - mB*rA*(cos(y1-y3))^2 )^(-
1) * ...
    ( -mB*rA*(y2^2)*sin(y1-y3)*cos(y1-y3) + mB*g*sin(y3)*cos(y1-y3) -
...
    mB*rB*(y4^2)*sin(y1-y3) - (mA+mB)*g*sin(y1) );

f4 = @(mA,mB,rA,rB,g,y1,y2,y3,y4) ( rB - (mB*rB/(mA+mB))*(cos(y1-y3))^2 )^(-
1) * ...
    ( rA*(y2)^2*sin(y1-y3) - g*sin(y3) + (mB*rB/(mA+mB))*(y4^2)*sin(y1-
y3)...
    *cos(y1-y3) + g*sin(y1)*cos(y1-y3) );

%% %% euler method
for i = 2 : length(t)
    y1(i) = y1(i-1) + h*y2(i-1);
    y3(i) = y3(i-1) + h*y4(i-1);
    y2(i) = y2(i-1) + h*f2(mA,mB,rA,rB,g,y1(i-1),y2(i-1),y3(i-1),y4(i-1));
    y4(i) = y4(i-1) + h*f4(mA,mB,rA,rB,g,y1(i-1),y2(i-1),y3(i-1),y4(i-1));
end

theta_A    = y1;
velocity_A  = y2;
theta_B    = y3;
velocity_B  = y4;

xA3 = rA*sin(theta_A);
yA3 = -rA*cos(theta_A);
xB3 = rA*sin(theta_A) + rB*sin(theta_B);
yB3 = -rA*cos(theta_A) - rB*cos(theta_B) ;

```

- By improved eulers method:

```

mA = app.MassAEditField_6.Value;
mB = app.MassBEditField_6.Value;
rA = app.LengthAEditField_6.Value;
rB = app.LengthBEditField_6.Value;
g = app.gEditField_4.Value;

theta_A0    = app.AngleofAEditField_6.Value*pi/180;
theta_B0    = app.AngleofBEditField_6.Value*pi/180;
velocity_A0  = app.AngularvelocityAEditField_2.Value;
velocity_B0  = app.AngularvelocityBEditField_2.Value;

time        = app.TimeEditField_4.Value;
framerate    = app.FramerateEditField_3.Value;

h = 1/framerate;

```

```

y1(1) = theta_A0;
y2(1) = velocity_A0;
y3(1) = theta_B0;
y4(1) = velocity_B0;

t = 0 : h : time ;

f2 = @(mA,mB,rA,rB,g,y1,y2,y3,y4) ( (mA+mB)*rA - mB*rA*(cos(y1-y3))^2 )^(-
1) * ...
    ( -mB*rA*(y2^2)*sin(y1-y3)*cos(y1-y3) + mB*g*sin(y3)*cos(y1-y3) -
mB*rB*...
    (y4^2)*sin(y1-y3) - (mA+mB)*g*sin(y1) );

f4 = @(mA,mB,rA,rB,g,y1,y2,y3,y4) ( rB - (mB*rB/(mA+mB))*(cos(y1-y3))^2 )^(-
1) * ...
    ( rA*(y2)^2*sin(y1-y3) - g*sin(y3) + (mB*rB/(mA+mB))*(y4^2)*sin(y1-
y3)...
    *cos(y1-y3) + g*sin(y1)*cos(y1-y3) );

for i = 2 : length(t)
    y1(i) = y1(i-1) + h*y2(i-1);
    y3(i) = y3(i-1) + h*y4(i-1);

    u2(i) = y2(i-1) + h*f2(mA,mB,rA,rB,g,y1(i-1),y2(i-1),y3(i-1),y4(i-1));
    u4(i) = y4(i-1) + h*f4(mA,mB,rA,rB,g,y1(i-1),y2(i-1),y3(i-1),y4(i-1));

    y2(i) = y2(i-1) + ...
    0.5*h*( f2(mA,mB,rA,rB,g,y1(i-1),y2(i-1),y3(i-1),y4(i-1)) +
f2(mA,mB,rA,...
    rB,g,y1(i),u2(i),y3(i),u4(i)) );
    y4(i) = y4(i-1) + ...
    0.5*h*( f4(mA,mB,rA,rB,g,y1(i-1),y2(i-1),y3(i-1),y4(i-1)) +
f4(mA,mB,rA,...
    rB,g,y1(i),u2(i),y3(i),u4(i)) );
end

theta_A    = y1;
velocity_A  = y2;
theta_B    = y3;
velocity_B  = y4;

xA2 = rA*sin(theta_A);
yA2 = -rA*cos(theta_A);
xB2 = rA*sin(theta_A) + rB*sin(theta_B);
yB2 = -rA*cos(theta_A) - rB*cos(theta_B) ;
end

```

Code for plotting and animation for the double pendulum:

```

axis(app.UIAxes_2, 'equal')
app.UIAxes_2.set('XLim', [-1.1*(rA+rB) 1.1*(rA+rB)])
app.UIAxes_2.set('YLim', [-1.1*(rA+rB) 1.1*(rA+rB)])
app.UIAxes_2.set('Color', 'k')

%% plot part
for i = 1:length(xA)
    cla(app.UIAxes_2);
    hold(app.UIAxes_2, "on") ;
    if app.skip == 1
        i = length(xA);
    end
    % Rod 1
    plot(app.UIAxes_2,[0 xA(i)], [0 yA(i)], 'w', 'LineWidth', 2)
    % Rod 2
    plot(app.UIAxes_2,[xA(i) xB(i)], [yA(i) yB(i)], 'w', 'LineWidth', 2)
    % Origin
    p = plot(app.UIAxes_2,0,0, 'wo');
    set(p, 'MarkerFaceColor', 'w', 'MarkerSize', 4)

    % Trajectory A
    plot(app.UIAxes_2,xA(1:i),yA(1:i), 'color', app.ColorDropDown_4.Value, 'LineWidth', 1)

    % Trajectory B
    plot(app.UIAxes_2,xB(1:i),yB(1:i), 'color', app.ColorDropDown_5.Value, 'LineWidth', 1)

    % Position A
    p = plot(app.UIAxes_2,xA(i),yA(i), 'r');

    set(p, 'Marker', 'o', 'MarkerFaceColor', app.ColorDropDown_4.Value, 'Color', 'k', 'MarkerSize', 10)
    % Position B
    p = plot(app.UIAxes_2,xB(i),yB(i), 'g');

    set(p, 'Marker', 'o', 'MarkerFaceColor', app.ColorDropDown_5.Value, 'Color', 'k', 'MarkerSize', 10)
    pause(0.00000001);
end
app.skip = 0;

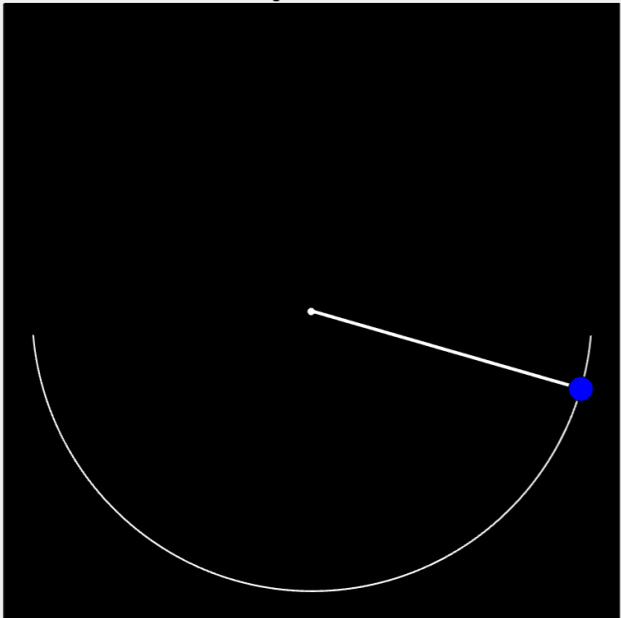
```

Output for some inputs :

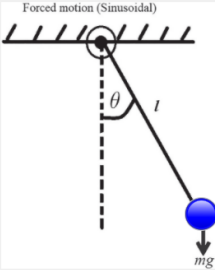
Here are some test cases and the output of the program:

Simple pendulum:

Single Pendulum



Forced motion (Sinusoidal)



Single Pendulum

l = Length

x = Angle in Degrees

m = Mass

It is suggested to keep the input values closer to the default ones , for better animation .

Color of the Ball

Blue

Mass

2

Length

0.1

g

9.8

Time

3

Frame Rate

200

Angle in Degrees

85

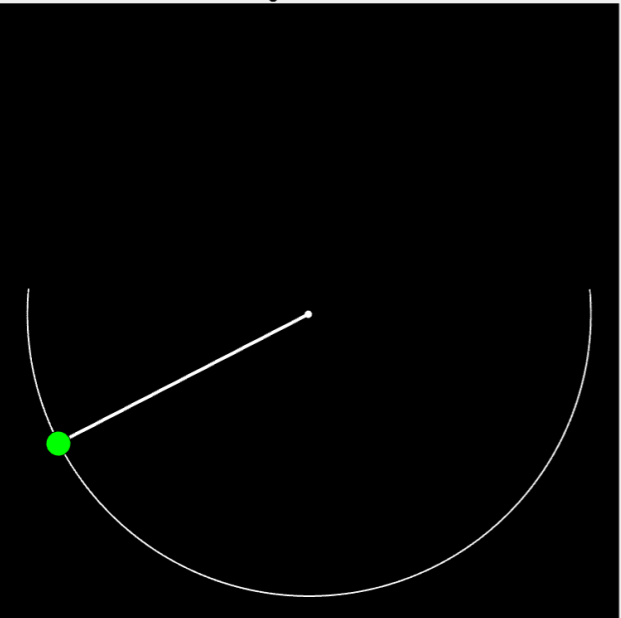
Angular velocity

0

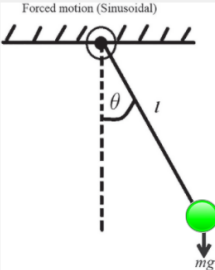
Skip

Run

Single Pendulum



Forced motion (Sinusoidal)



Single Pendulum

l = Length

x = Angle in Degrees

m = Mass

It is suggested to keep the input values closer to the default ones , for better animation .

Color of the Ball

Green

Mass

20

Length

15

g

9.8

Time

6

Frame Rate

200

Angle in Degrees

95

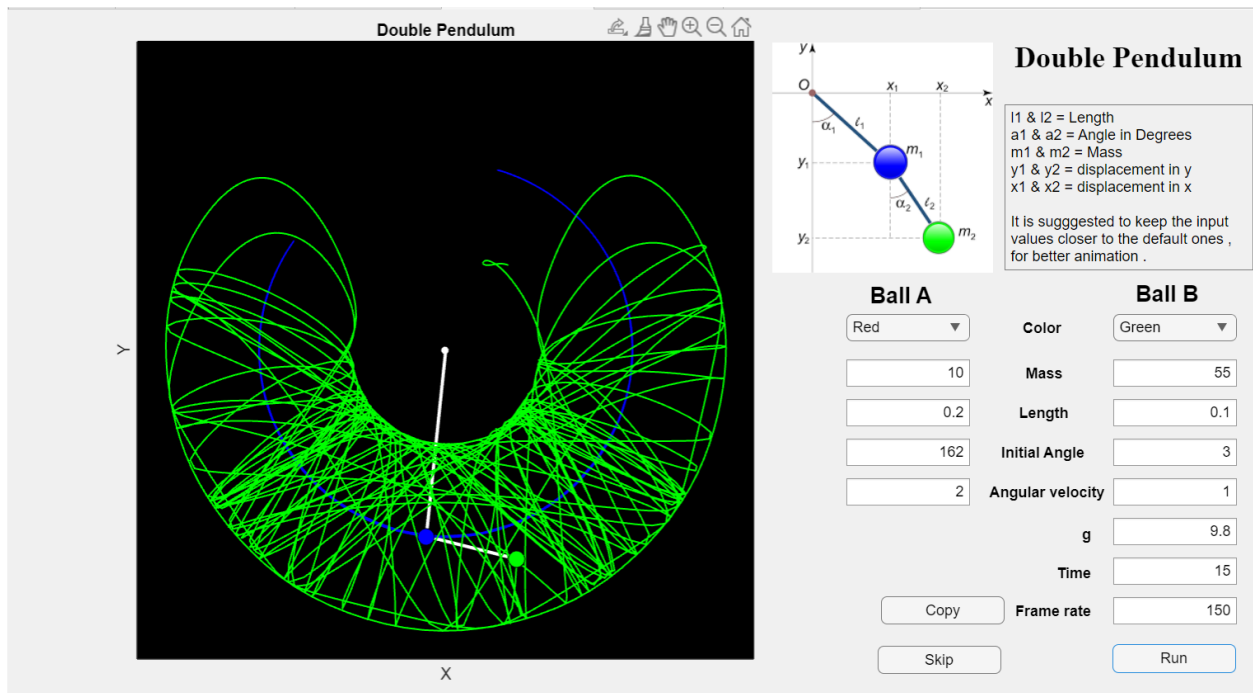
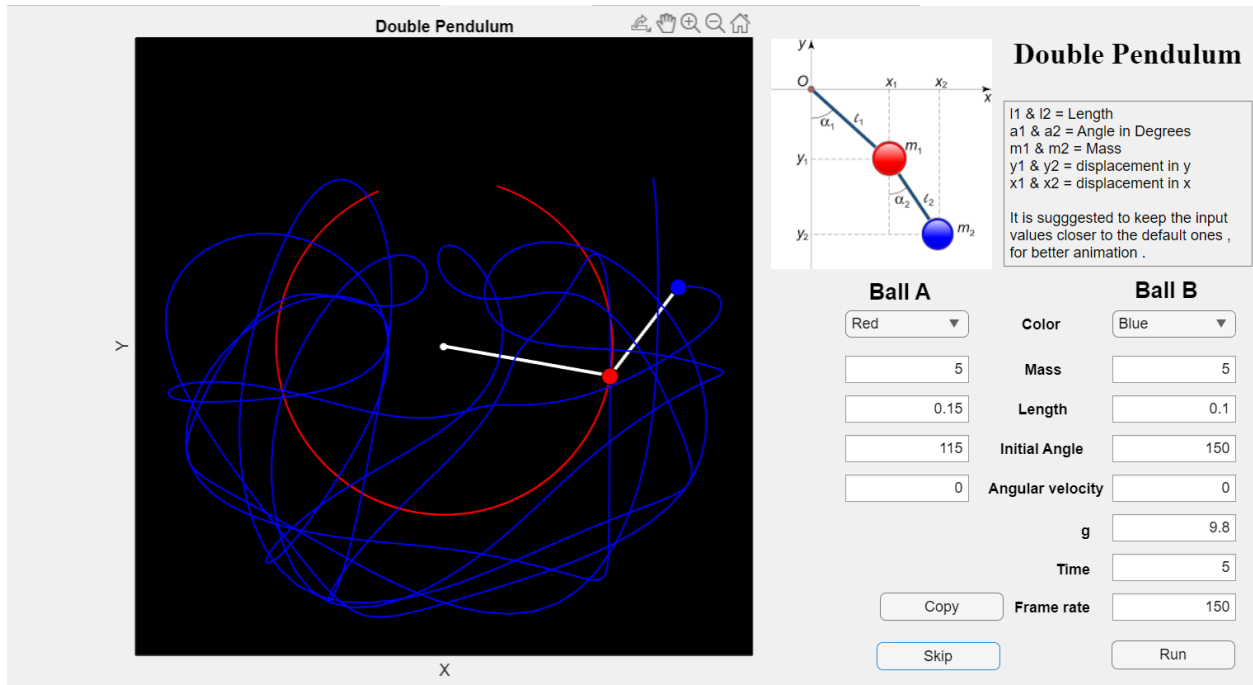
Angular velocity

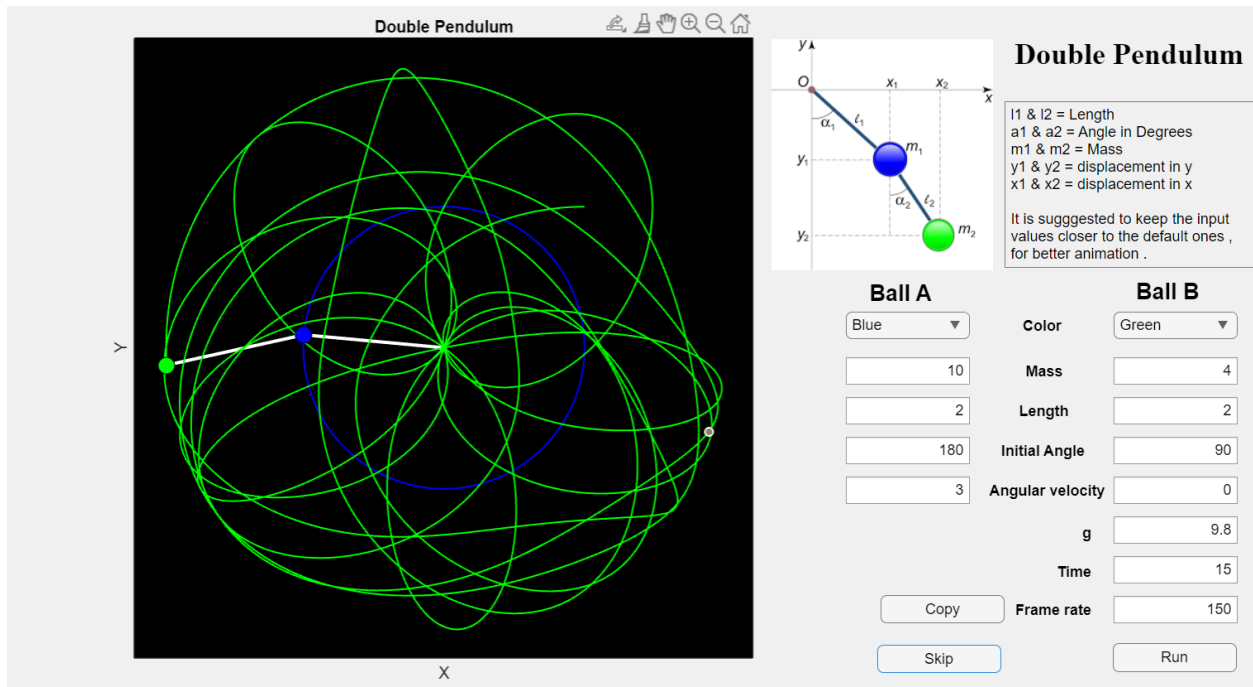
0

Skip

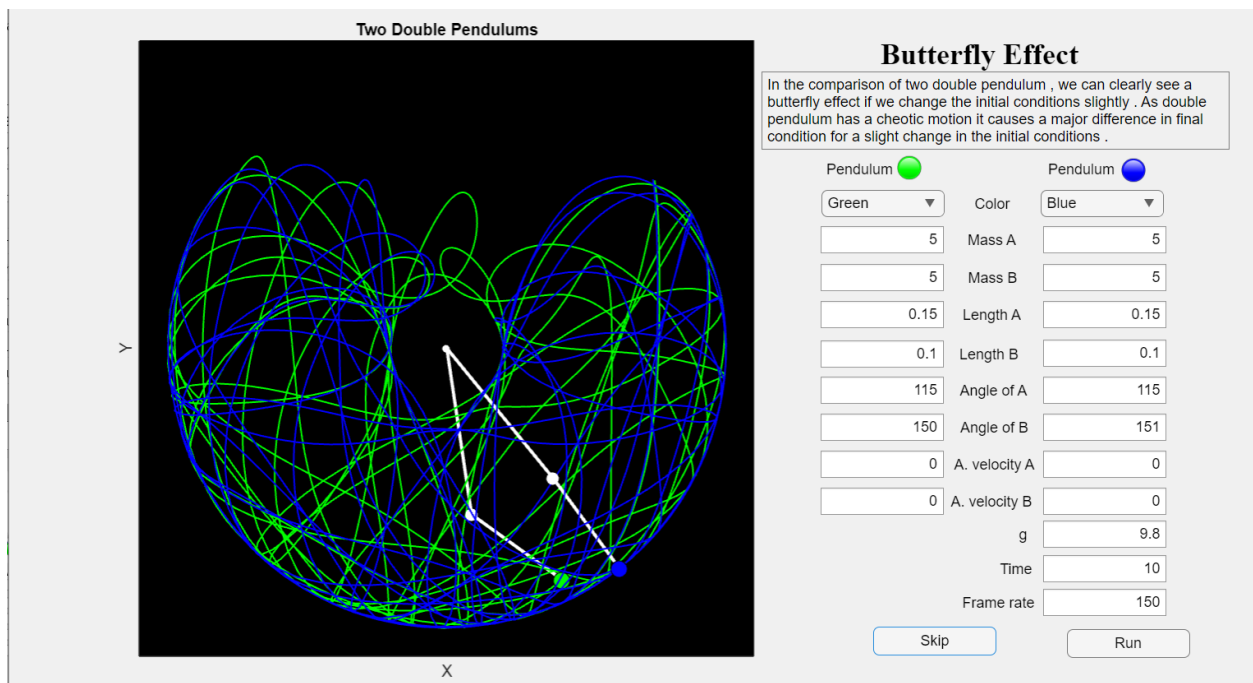
Run

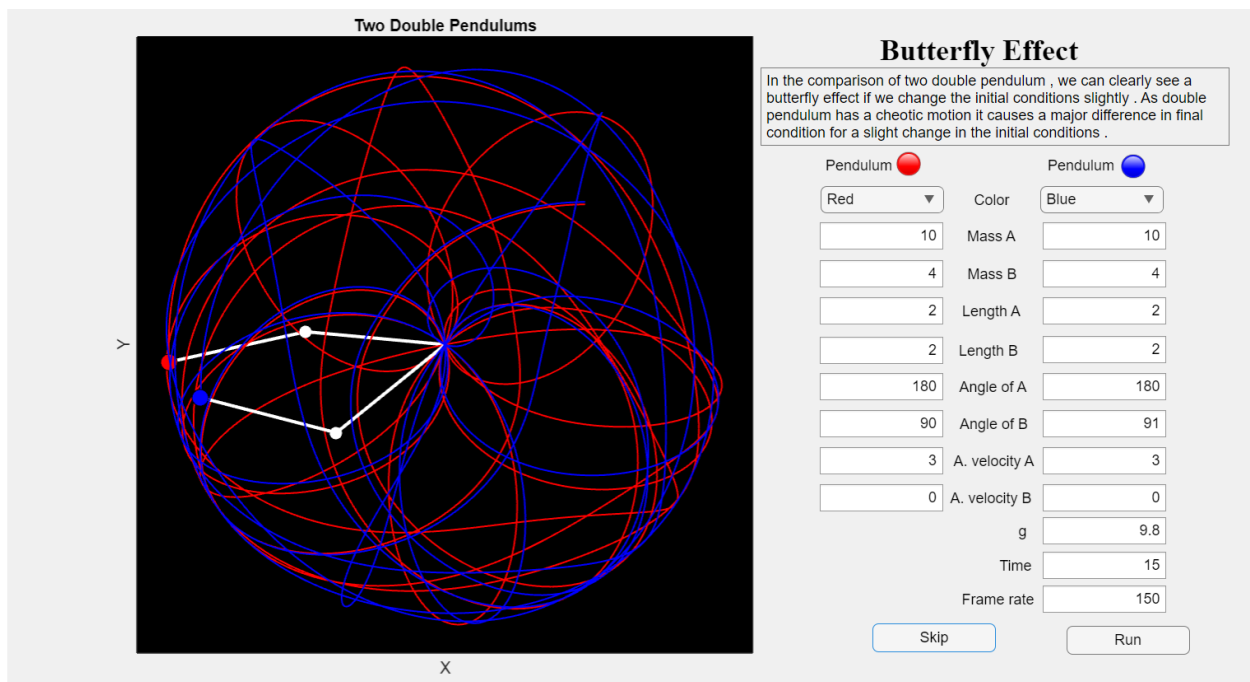
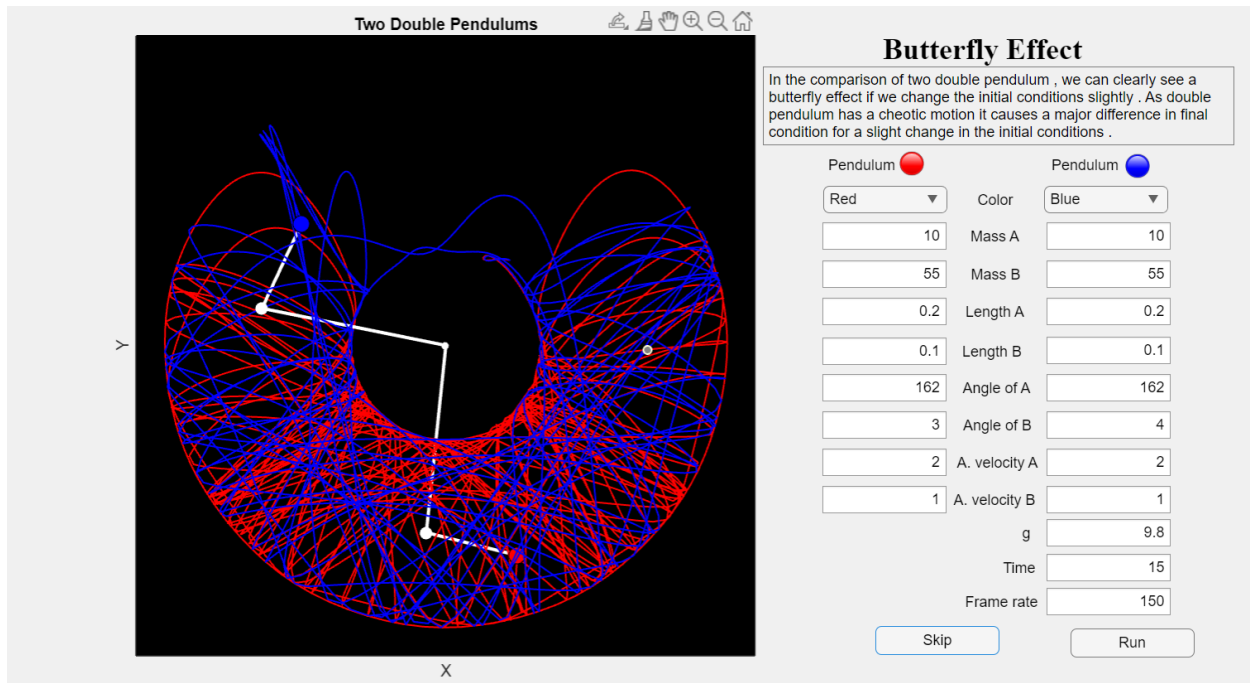
Double pendulum:



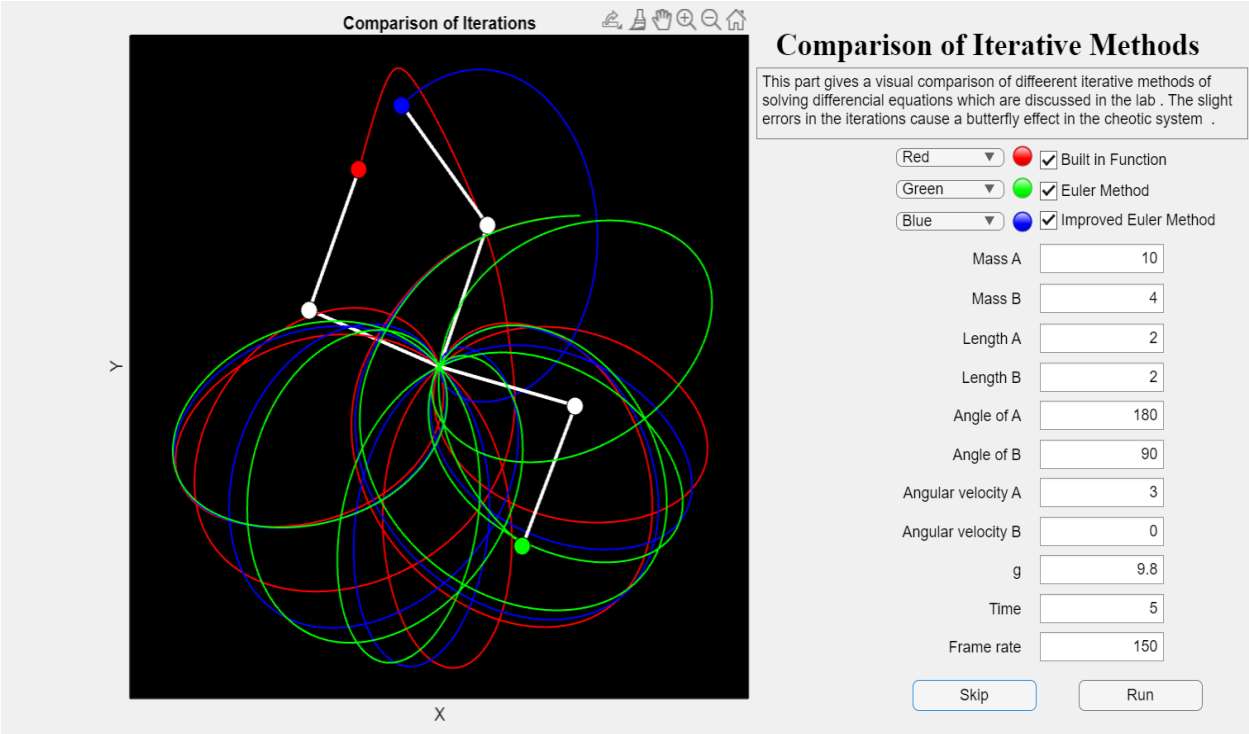
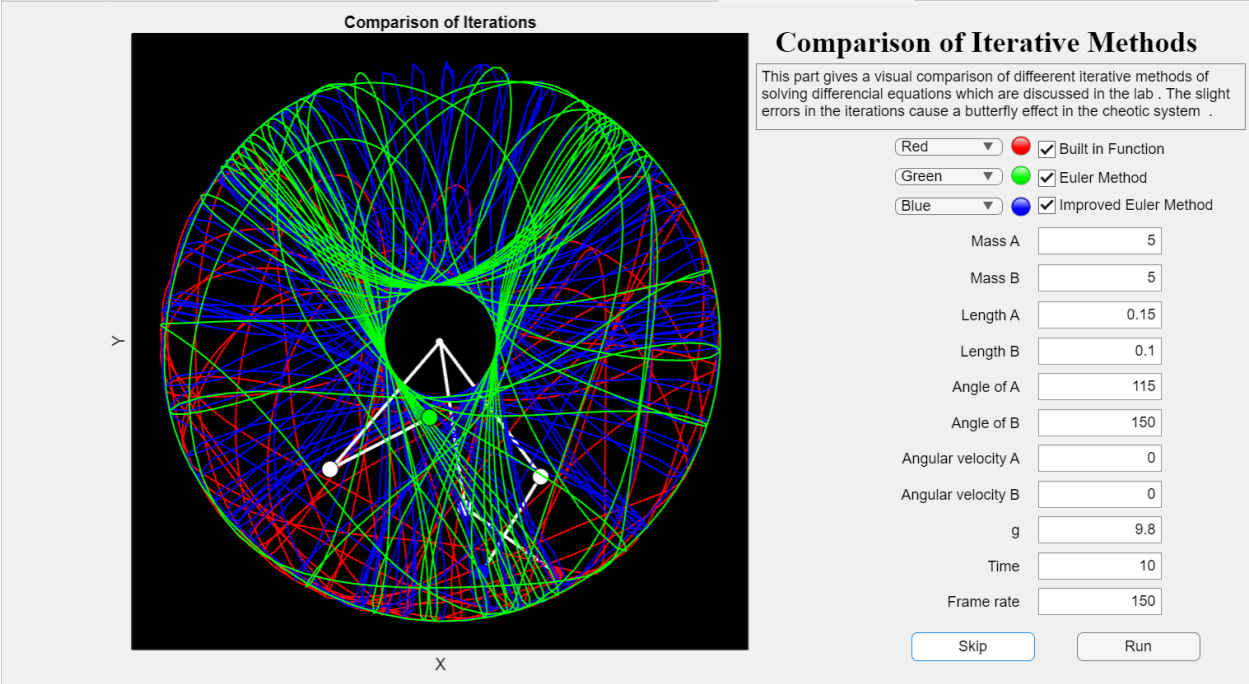


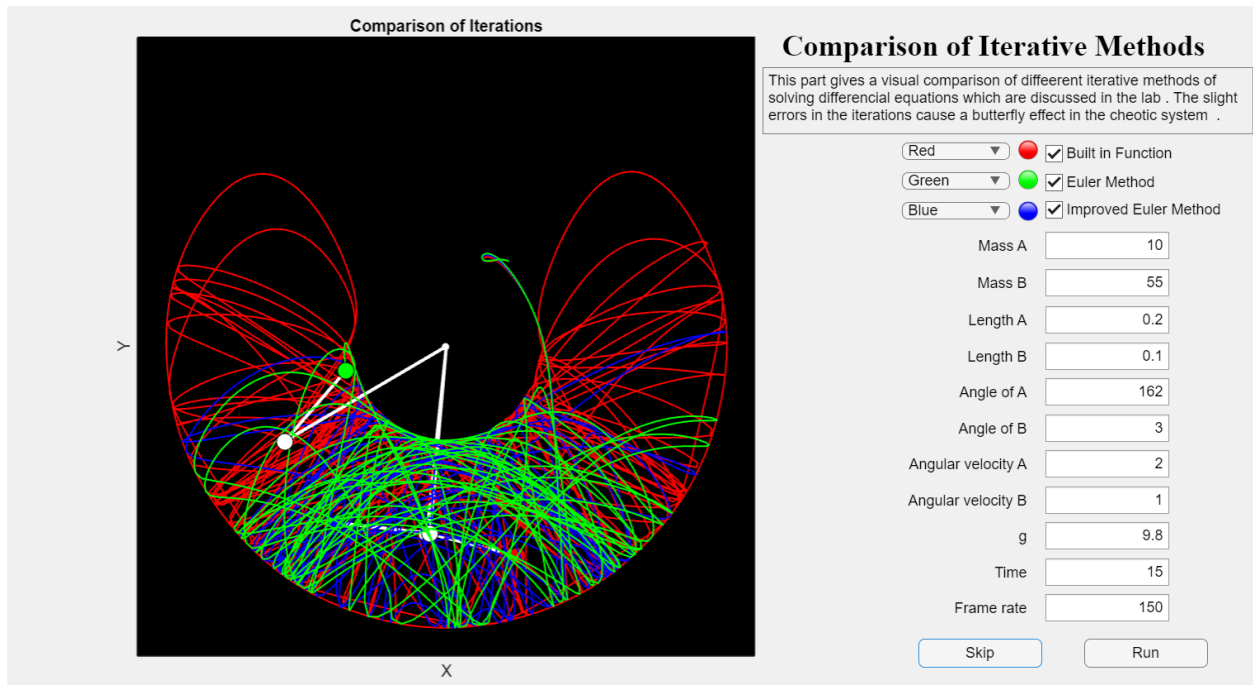
Two double pendulum and butterfly effect:





Different iteration methods and butterfly effect :





Contribution:

Both of the developers understood the code but it was mainly written by 2006054 (Joyonta Kartik Debnath). And, the app designing was done simultaneously by both of them. Thus, the contribution of ID: 2006054 will be greater in consideration.

Code link:

https://drive.google.com/drive/folders/1f2JLcFgv4vvH4HtaMjzE491_ZKbGW1jv?usp=s
haring